

專題題目

音訊轉換簡譜系統

專題學生：鄒芷瑄
龔正媛
郭妍妮

指導教授：卓世明 教授



大同大學
資訊工程學系
專題報告

中華民國 115 年 1 月

音

訊

轉

換

簡

譜

系

統

大

同

資

工

系

專

題

報

告

龔 鄒 郭

正 芷 妍

媛 瑄 妮

大同大學
資訊工程學系
專題報告
音訊轉換簡譜系統

鄒芷瑄

龔正媛

郭妍妮

經考試合格特此證明

專題考試委員

指導教授

中華民國 115 年 1 月 6 日

目錄

摘要

第1章 概述：專題簡介，專題動機，專題目標，專題成果概述

第2章 文獻探討

第3章 系統設計與架構

第4章 系統實作

第5章 結論與未來發展

參考文獻

摘要

本專題旨在解決音樂教育與內容製作領域中，將音樂檔案自動轉換為數字簡譜的高成本和複雜性問題。

我們開發了一個端到端的自動化轉譯系統，能夠將使用者上傳的 MP3 音樂，直接轉換為專業排版的 PDF 數字簡譜。

第一章

概述

1.1 專題簡介

音源分離（Source Separation）技術近年來隨著深度學習快速發展，其中 Facebook AI Research 團隊所提出之（Deep Extractor for Music Sources）模型，具備處理時域（waveform）音訊的能力，且於 v3 版本中引入 Hybrid 結構，結合時域與頻域特徵以提升人聲分離品質。

Défossez 等人於論文 “Hybrid Demucs: Scalable Hybrid and Multi-Scale Architecture for Music Source Separation”（arXiv:2111.03600v2）[1]中，證實該模型在 MUSDB18 等資料集上表現優於傳統頻譜型模型如 Open-Unmix，並成功支援人聲、鼓、貝斯與其他音源的分離，為多階段音樂處理系統奠定基礎。

在人聲音訊轉譯為 MIDI 的部分，Spotify 研究團隊所提出之 Basic Pitch 模型（Bittner et al., 2022；arXiv:2203.09893v2）[2]為一輕量級、多樂器適用的音符轉譯系統，可從單聲道或多聲部音訊中同時預測音高（pitch）、音符起始（onset）與音符啟動狀態（activation），有效提升轉譯準確率。該模型特別支援 vibrato 與 pitch bend 等表現細節，並具備低資源運算需求，能於瀏覽器或行動裝置即時執行，適合應用於 Demucs 分離後之人聲進行 MIDI 音符標註與時間對齊。

然而，為了將音符資訊轉換為具備正確樂理結構的樂譜格式，本研究引入開源製譜軟體 MuseScore 作為關鍵的轉檔核心。利用 MuseScore 強大的指令列

介面 (Command Line Interface, CLI)，本系統能將 Basic Pitch 產出的 MIDI 檔案自動轉換為包含完整樂譜資訊的 MusicXML 標準格式，藉此來解決音符量化與小節劃分的複雜問題。隨後，透過整合 musicxml_to_jianpu 與 jianpu-ly 等工具，將 MusicXML 解析並轉譯為 LilyPond 語法。

總結而言，本專題建構了一個完整的端到端自動化管線：結合 Demucs 進行音源分離、Basic Pitch 進行人聲轉譯，利用 MuseScore 進行標準化樂譜轉換，最終透過 LilyPond 排版引擎輸出。此系統能將 MP3 音樂自動轉換為可視化的數字簡譜，最終輸出印刷等級的樂譜，廣泛應用於音樂教育、跨文化樂譜轉譯與個人創作輔助等場景。

1.2 專題動機

目前市面上的主流製譜軟體多注重於五線譜的排版，缺乏對數字簡譜的專業支援與自動化轉換功能。對於音樂教育工作者，特別是歌唱指導老師而言，教材的取得往往面臨巨大挑戰。歌譜若另請專業人士聽寫製譜，則需耗費大量時間反覆聆聽、修音與排版，導致製作成本高昂且利潤微薄。因此，我們希望開發一套低成本、高效率的轉譜系統，讓使用者能付出更少的金錢與時間成本產出簡譜，利用 AI 達到省錢省時間的成品效益。

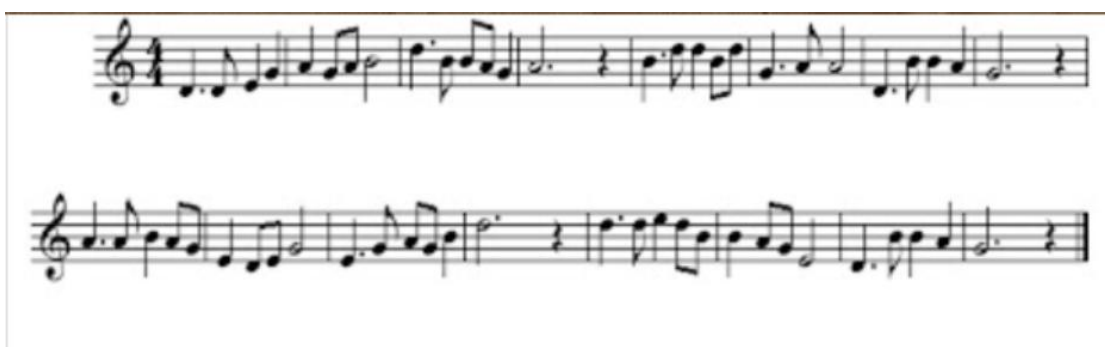


圖 1.1:五線譜:需要理解大部分規則才看懂

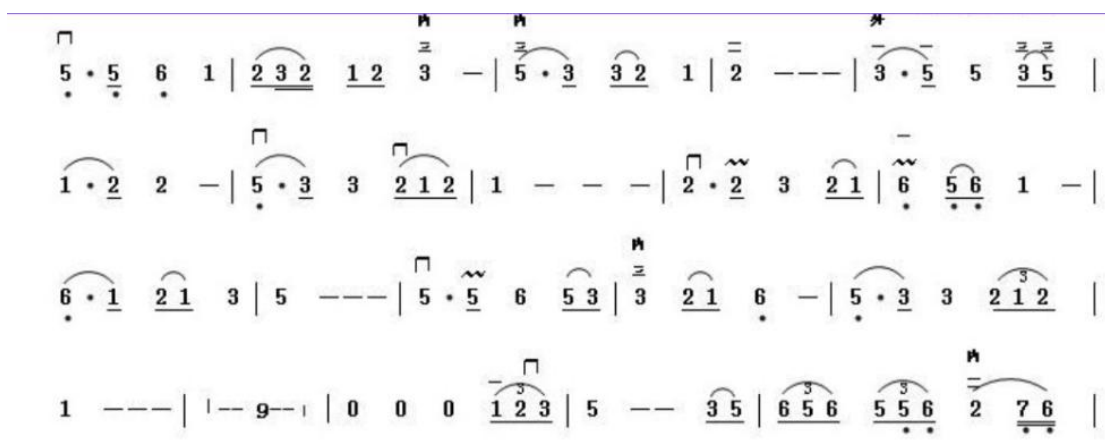


圖 1.2: 簡譜，數字呈現音高更易於初學者學習

1.3 專題目標

本專題旨在開發一個整合性的網頁平台，以實現從音訊上傳到簡譜輸出的全自動化流程。首要目標是建構支援多種格式（如 MP3、WAV）的音訊處理核心，使系統能自動執行後端的音源分離與轉譯任務。鑑於人工智慧轉譯在複雜樂句中可能存在的誤差，本系統亦致力於提供直觀的可視化微調介面，允許使用者在最終輸出前，針對 MIDI 音高進行人工檢視與校正，以確保樂譜的準確性。最終目標則是實現高度自動化的排版輸出，使用者僅需輸入速度（BPM）等基本參數，系統即可串接 MuseScore 與 LilyPond 引擎，快速生成符合標準格式且美觀的 PDF 數字簡譜，供使用者下載與應用。

1.4 專題成果概述

本研究成功實作了一個整合音訊處理後端與網頁前端的轉譯系統。使用者透過瀏覽器介面將 MP3 音檔上傳後，系統後端即自動呼叫 Demucs 去除背景音樂，保留純淨人聲，並透過 Basic Pitch 轉譯為 MIDI。隨後，系統自動化呼叫 MuseScore 將 MIDI 轉為 MusicXML，再經由轉碼腳本生成簡譜語法。

在編輯介面上，使用者可依照需求調整音高，提高準確性。最終，系統透過自動化排版引擎，成功產出一份排除背景干擾、專注於人聲旋律的 PDF 數字簡譜，驗證了從音訊輸入到樂譜輸出的完整可行性。



圖 1.3:音訊轉換簡譜系統

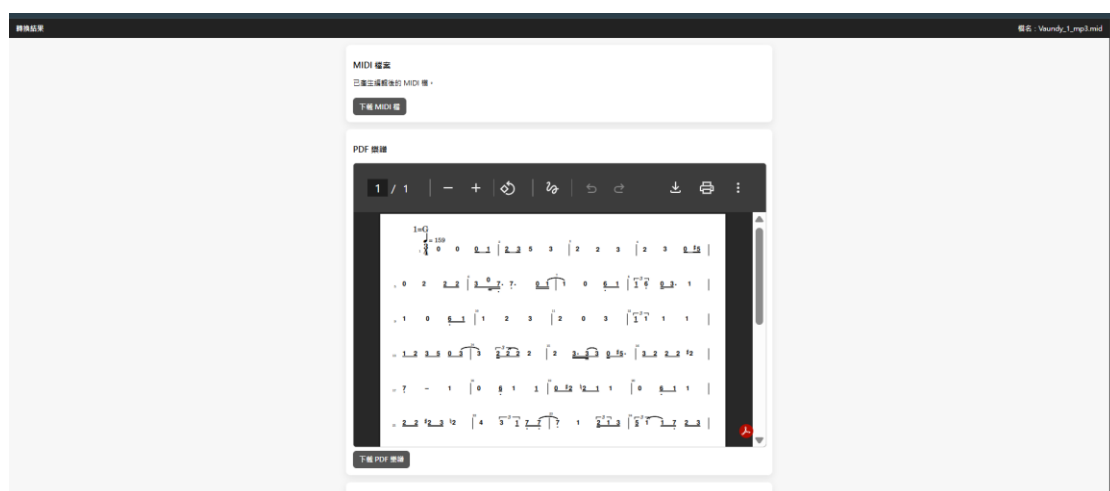


圖 1.4:生成之簡譜輸出預覽（嵌入式檢視器）。

第二章

文獻探討

2.1 Demucs

2.1.1 概述

音源分離作為音訊處理領域的重要研究方向，傳統方法大多數依賴短時傅立葉轉換（STFT）來獲取頻域資訊，但因忽略相位或重建失真，常造成還原音訊品質下降。

針對此問題，Facebook AI Research 團隊提出 Demucs 系列模型，於 2021 年發表的論文”Hybrid Spectrogram and Waveform Source Separation”（Défossez et al., 2021）[1]中提出一種結合 waveform 與 spectrogram 的雙路徑架構。該模型不僅保留了 waveform 模型還原音質自然的優點，亦透過頻域路徑增強分離清晰度，並以 weighted sum 融合輸出結果，在 MUSDB18-HQ 資料集上實測 SDR 表現優異，超越先前方法如 Open-Unmix 與 Spleeter。整體而言，Demucs 具備訓練穩定性高、應用彈性大與結果音質自然等特點，適合作為本研究中音源分離的核心技術。

在音源分離的眾多技術中，Demucs 具備難以取代的優勢。首先，傳統的分離工具多依賴於頻譜域（spectrogram-based）操作，雖然計算效率較高，但容易造成相位錯誤與音質劣化。而 Demucs 採用 waveform-based 深度學習模型，在保留音質自然性方面更具優勢。

更進一步，Hybrid Demucs（Défossez et al., 2021）[1]結合了 waveform 與 spectrogram 的優點，讓模型能針對不同音源自動選擇最佳分離域。這種自適應

能力在當前音源分離技術中極為少見，也正是該模型能於 Sony 主辦之 Music Demixing Challenge 2021 奪冠的關鍵。

此外，根據該論文實驗結果，Demucs 在 MUSDB HQ 資料集上的平均 SDR 提升達 1.4 dB，遠超其他主流工具。配合主觀聽感評比，Demucs 不僅分離精度高，也能有效抑制「聲音串音」(bleeding)，顯示其在真實音樂處理中亦具實用性。

雖然模型較大、推論成本高，但若研究或應用目的是高品質音樂處理、精準的樂器分離或聲音分析，則 Demucs 提供的音質保真度與技術領先性，遠勝過傳統 Spleeter、Open-Unmix 等工具，是目前最具代表性的音源分離方案之一。

2.1.2 模型優點

分離品質優異，表現領先: Hybrid Demucs 在 MUSDB HQ 資料集中達到平均 SDR 提升 1.4 dB，並在 Sony 主辦的 Music Demixing Challenge 2021 中奪冠，證明其領先同類模型的準確度與穩定性。

音質自然、主觀評分佳: 在主觀評估中獲得整體音質 7.68/10 的分數，污染控制 (absence of bleeding) 得分達 3.04/5，顯示分離結果不僅準確，也更接近人耳所期待的自然感與純淨度。

支援多聲源與高音質音樂處理: 能處理 vocals、drums、bass、others 等 4 類音源，且支援高取樣率 (44.1kHz stereo) 音訊，適用於真實音樂與製作環境。架構彈性高，可進一步優化: 模型結構模組化，支援 fine-tuning、ensemble、bagging 與其他進階訓練策略，具備良好的擴展性與研究潛力。

開源、擴展性強，適合學術與實作: Demucs 擁有完整的開源架構與預訓練

模型，使用者可直接下載使用，也可自訂訓練資料進行再訓練（fine-tune），支援四類音源（人聲、鼓、貝斯、其他），並且提供多模型平均（ensemble）以提升穩定性。此外，其模型架構與訓練流程皆有完整論文與程式碼對應，便於學術研究引用與深入修改。

2.1.3 限制與挑戰

推論成本高、模型龐大: 需要同時運行多個分支（waveform + spectrogram）與集成策略，對硬體資源需求高

模型結構複雜、調整成本高: 結構包含 LSTM、local attention、residual blocks 等，雖性能佳但實作門檻較高。

部分聲源仍有交叉污染現象: 雖然表現優異，但在使用者主觀評估中仍低於 ground truth，顯示在極複雜混音情境中仍可能出現聲源混疊。

泛化能力尚未全面驗證: 模型訓練與測試皆基於 MUSDB HQ 等公開資料集，對未知類型聲源（如特殊樂器、語音）之泛化能力仍待觀察。

2.1.4 總結

Demucs 具備高音質輸出、雙分支融合架構、強大開源社群與完整文獻支持，是目前最先進的音源分離技術之一。無論在主觀聽感、實驗指標，或是可實作性上，都優於傳統頻譜模型如 Spleeter、Open-Unmix。

Demucs 已能提供高音質與低串音的音源分離成果，具備良好的實用性與效率。也因此，我們選擇以 Demucs 系列為分離模型基礎，代表的是一套經過驗證、具擴展潛力、可適應實際應用需求的架構族群。所以我們選擇 Demucs 作為音源分離核心技術，具備明確的理論基礎與實證支持。

2.2 Basic Pitch

2.2.1 概述

為實作將人聲音訊轉換為 MIDI，本研究採用 Spotify 團隊提出的 Basic Pitch 模型 (Bittner et al., 2022) [2]，該模型具備樂器無關性、輕量化、高準確度等特性，能有效處理經 Demucs 分離後之人聲訊號，輸出包含音高、起始與持續資訊的多音音符資訊。其支援 vibrato、pitch bend 等音高變化，並可透過簡單轉換輸出數字簡譜。

自動音樂轉譯 (Automatic Music Transcription, AMT) 被視為一項關鍵技術，具備廣泛應用潛力。由於此任務本身極具挑戰性，目前最佳成果多來自於針對特定場景所設計的系統。然而，這些「專用系統」雖然準確率高，卻難以應用於現實場景。儲存與網路限制無法支援同時使用多套模型，而記憶體與執行時間限制也使得模型複雜度受到壓抑。

因此，本研究中提出一種輕量神經網路架構，用於音樂樂器轉譯（包含人聲），支援複音 (polyphonic) 輸出，並能廣泛應用於各類樂器音訊。模型同時預測三種 frame-level 資訊：音符起始 (onset)、多音音高 (multipitch) 與音符啟動狀態 (activation)。實驗證實，這種多任務結構能提升音符層級的準確性。

2.2.2 模型優點

低資源需求: 作者設計模型的動機是為了解決儲存與網路限制 (storage and network constraints) 和記憶體與執行時間限制 (memory and run-time constraints)，表明了該模型是針對「低資源場景」設計。

此研究所提出的模型是一個輕量級神經網路 (lightweight neural network)，能在低階設備上高效運行，歸功於其低記憶體與低處理時間需求，不需重新訓

練就泛化至各種複音（polyphonic）或單音（monophonic）樂器，並且足夠輕量，可以在低資源的環境中運行。同時考量了運行速度與推論期間的記憶體高峰使用量，並刻意限制模型架構的深度，以降低記憶體需求。需要注意的是，模型參數的數量並不總是與記憶體使用量呈現正比；例如捲積層參數量雖少，但由於特徵圖尺寸大，仍可能產生高記憶體消耗。

模型為一個完全由捲積構成的模型，總參數量僅有 16,782 個（能在記憶體有限的環境中順暢運行）。

延續核心架構與優勢：Basic Pitch 為 Spotify 於 2022 年開源的音符轉譯工具，其架構與 NMP 幾乎相同，亦採用 harmonic stacking 與輕量化的卷積模型，繼承了 NMP 在效率與可移植性上的優點。

實務可用性高：Basic Pitch 提供完整的推論程式與訓練模型，支援即時應用、跨平台部署（如 Web 應用或 Raspberry Pi），降低開發門檻；相較於 MI-AMT 或 Vocano，Basic Pitch 不需繁瑣設定即可使用，適合學術研究與實務整合。

具備 MPE 與音符輸出雙重功能：延續 NMP 的設計，Basic Pitch 同樣同時輸出 multipitch 與 note event，可同時應用於音準偵測與音符轉譯任務，增加彈性。

穩定開源社群支援與 Spotify 維護：Basic Pitch 作為 Spotify Research 支持之專案，文件完整、社群活躍，較無需擔心模型棄用或缺乏更新問題。

不犧牲精度的前提下保持低資源消耗：即便不如專門為特定樂器調整的模型（如 Onsets and Frames、Vocano）在個別任務上極致精準，但對於泛用性應用如轉譯任意人聲或樂器的旋律線來說，Basic Pitch 在精度與計算效率間取得

良好平衡。

2.2.3 限制與挑戰

受限於 onset 預測的準確度: 音符建立仰賴 heuristics 方法進行 peak picking 與時間追蹤，並非端到端訓練而得

不如鋼琴與人聲的專業模型: 與專用樂器模型相比，NMP 在 GuitarSet 上達到最先進水準；但在 鋼琴與人聲 上則無法超越專用模型

未使用 pruning/compression 技術進一步減重: 初衷為輕量，但尚未進行壓縮技術處理

2.2.4 總結

Basic Pitch 是一個在多樂器、多情境下皆可運作的「實用」選擇，尤其適合本研究目標：開發一個可即時使用、運算負擔低、轉換結果穩定的音高分析或簡譜系統。

2.3 MuseScore

MuseScore 是一款開放原始碼的音樂譜寫軟體，支援多種樂譜編輯與排版功能，並廣泛應用於音樂教育與作曲編輯等領域。其所提供之圖形化操作介面採用即時所見即所得（What You See Is What You Get, WYSIWYG）設計，使使用者能夠直觀地進行樂譜編輯與檢視。廣泛應用於音樂教育、作曲與編曲領域。該軟體支援多種作業系統，並具備強大的樂譜渲染引擎，能夠處理複雜的樂理記號與排版需求。除了圖形介面操作外，MuseScore 提供了指令列介面（Command Line Interface, CLI）功能，使其能夠在無須開啟視窗的情況下，執行批次轉檔與自動化處理任務。

在本研究的自動化管線中，MuseScore 扮演著至關重要的「中介標準化」角色。由於前述 Basic Pitch 模型產出的 MIDI 檔案僅包含音符的觸發時間（Note On/Off）與音高資訊，缺乏小節線、拍號與結構化的樂理邏輯（如連音、附點的正確歸類）。若直接對原始 MIDI 進行解碼，容易因 AI 預測的微小時間誤差導致節奏判讀混亂。

因此，本系統利用 MuseScore 成熟的量化（Quantization）演算法，透過 CLI 指令將 MIDI 檔案轉換為 MusicXML 格式。MusicXML 是一種基於 XML 的標準開放格式，專門用於描述數位樂譜結構。透過 MuseScore 的轉換，原始的 MIDI 訊號得以被「規整」為具備正確小節劃分與音符時值的結構化資料，為後續的簡譜編碼程序提供了準確且標準的輸入源，大幅提升了最終樂譜的可讀性與準確度。

2.4 jianpu-ly

為了提升簡譜（Jianpu）自動化排版的便利性，部分研究者與開源開發者提出了對 LilyPond 音樂排版系統的延伸方案。ssb22 等人於 GitHub 開源的 jianpu-ly 工具[4]，即為一個能將簡譜轉換為 LilyPond 語法的 Python 腳本，支援多種語法並整合至原生排版流程中，避免手動處理五線譜轉換問題。該工具透過在特殊自定義的譜表上繪製簡譜，讓使用者能利用 LilyPond 原有的排版能力（如連音線、自動間距、歌詞對齊等），維持編排一致性。jianpu-ly 亦提供 MusicXML 匯入功能，雖仍為實驗性質，卻顯示其潛在擴充性。此一工具對於研究中需產出標準化簡譜格式者提供實用價值。

2.5 LilyPond

雖然本研究主要聚焦於深度學習技術（Demucs 與 Basic Pitch）實現音源分離與

MIDI 轉譯，但最終輸出視覺化樂譜的品質，則高度仰賴專業的音樂排版系統。Han Wen Nienhuys 與 Jan Nieuwenhuizen 於 2003 年發表的論文 "LilyPond, a system for automated music engraving" [5] 中，闡述了 LilyPond 作為自動化音樂雕版系統的核心原理，強調其通過精密演算法，將樂譜資料轉換為符合傳統音樂美學標準的印刷級輸出。

由於現有主流製譜軟體在數字簡譜的自動化排版上仍有侷限，本研究最終確立了以 LilyPond 為核心的渲染策略。不同於過往研究嘗試直接對不穩定的原始 MIDI 進行編碼，本系統首先引入 MuseScore [3] 處理前端的音符量化與 MusicXML 標準化，隨後透過 jianpu-ly [4] 將結構化的 XML 資料精確轉譯為 LilyPond 語法。此一流程設計有效地將 LilyPond 定位為系統的最終專業渲染核心，確保經處理後的音訊資訊，無論是音高、音值、休止符或連音記號的呈現，都能達到高品質、視譜友善的印刷等級標準，從而大幅提升整體系統的實用性與專業價值。

第三章

系統設計與架構

3.1 系統架構

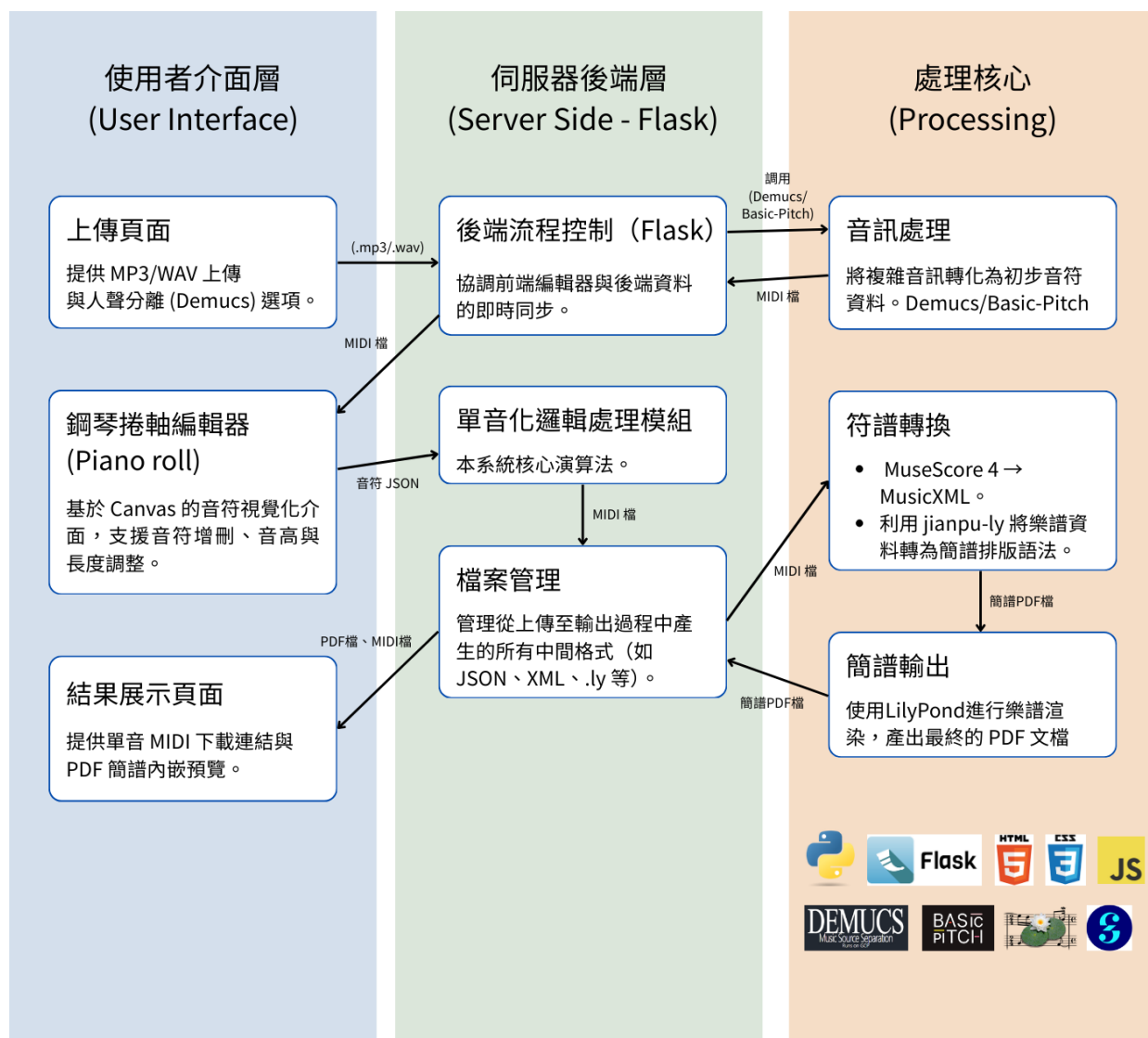


圖 3.1: 系統架構圖

本系統採用三層式架構設計，依功能劃分為**使用者介面層（User Interface）**、**伺服器後端層（Server Side – Flask）**與**處理核心層（Processing）**，以提升系統之模組化程度與維護性。整體架構如圖 3.1 所示。

使用者介面層負責與使用者互動，包含音檔上傳頁面、鋼琴捲軸（Piano Roll）編輯介面以及結果展示頁面。使用者可於上傳頁面提供 MP3 或 WAV 音檔，並選擇是否啟用人聲分離功能；由於 Basic Pitch 產生之 MIDI 仍可能包含雜音或音符誤差，因此系統特別提供鋼琴捲軸介面，讓使用者能以視覺化方式進行修正。鋼琴捲軸介面則提供音符的視覺化呈現，允許使用者調整音高、音符位置與長度，以修正自動轉換過程中可能產生之誤差；結果展示頁面則用於顯示最終輸出的簡譜 PDF，並提供單音 MIDI 檔案下載。

伺服器後端層以 Flask 作為核心，負責接收前端請求並進行流程控制與資料串接。後端除管理檔案上傳與中間檔案儲存外，亦負責協調音訊轉換流程，並在使用者完成音符編輯後，接收音符資料（JSON 格式）進行後續處理。此外，系統於後端實作單音化邏輯處理模組，針對可能出現之重疊音符進行修正，以確保後續簡譜輸出符合單旋律表示之需求。

處理核心層負責實際之音訊分析與樂譜轉換作業。音訊處理部分透過 Demucs(選配) 進行人聲分離，並使用 Basic Pitch 將音訊資料轉換為初始 MIDI 檔案；符譜轉換部分則利用 MuseScore 將 MIDI 轉換為 MusicXML，再經由簡譜轉換工具與 LilyPond 進行排版，最終輸出簡譜 PDF 文件。各模組之分工設計使系統能有效整合多種外部工具，並維持清楚的資料流向。

3.2 系統流程圖

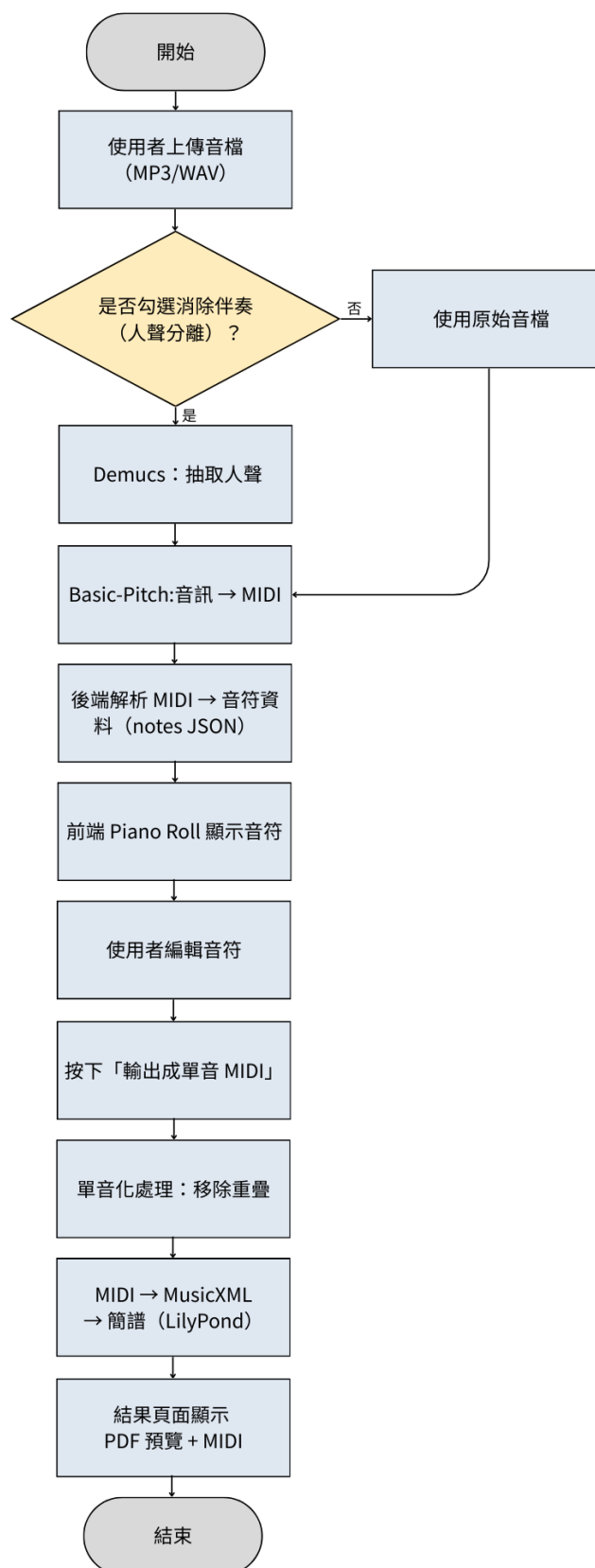


圖 3.2: 系統流程圖

本系統之整體運作流程如圖 3.2 所示，流程由使用者上傳音檔開始，最終輸出簡譜 PDF 與單音 MIDI 檔案。以下依流程順序進行說明。

首先，使用者於系統介面上傳欲處理之音檔，系統支援 MP3 與 WAV 兩種常見音訊格式。上傳完成後，系統會依使用者設定判斷是否啟用人聲分離（消伴奏）功能。若使用者勾選消伴奏選項，系統將透過 Demucs 模型對音訊進行人聲抽取，以取得較為純淨之人聲音訊；若未啟用此功能，則直接使用原始音檔進行後續處理。上述兩種情況最終皆會將音訊資料送入旋律抽取階段。

接著，系統使用 Basic Pitch 模型對音訊進行旋律分析，將音訊轉換為初始 MIDI 檔案，內容包含音符之音高與時間資訊。由於自動旋律抽取可能受到伴奏或背景雜音影響，所產生之 MIDI 檔仍可能存在音符誤差，因此後端會先解析 MIDI 檔案，將音符資料轉換為結構化之 notes JSON 格式，供前端進行顯示與編輯。

在前端部分，系統以鋼琴捲軸（Piano Roll）介面將音符資料視覺化呈現，使用者可透過此介面檢視旋律結構，並進行音符移動、音高調整、音符長度修改，以及新增或刪除音符等操作，以修正自動轉換過程中產生之不準確音符。完成編輯後，使用者按下「輸出成單音 MIDI」按鈕，系統即將編輯完成之音符資料傳送回後端。

後端在接收到音符資料後，會先執行單音化處理模組，針對可能存在之重疊音符進行修正。該模組會依音符時間順序檢查是否有重疊情形，並透過截斷

前一音符的方式，確保任一時間點僅存在單一音符，以符合單旋律與簡譜輸出的需求。單音化處理完成後，系統會輸出新的單音 MIDI 檔案。

最後，系統進行樂譜轉換流程，先將 MIDI 檔案轉換為 MusicXML 格式，再透過簡譜轉換工具與 LilyPond 進行排版處理，產生最終之簡譜 PDF 文件。完成後，系統於結果頁面提供簡譜 PDF 預覽，並同時提供單音 MIDI 檔案下載，供使用者保存與後續使用，至此整體流程結束。

3.3 前後端資料互動與資料格式設計

本系統採用前後端分離之架構設計，透過明確的資料格式與介面定義，使使用者介面與後端處理模組能有效進行資料交換與流程控制。本節將說明系統前後端之互動方式，以及各階段所使用之資料格式設計考量。

在音訊轉換流程中，後端首先接收使用者上傳之音檔，經旋律抽取後產生 MIDI 檔案。由於 MIDI 檔案本身屬於二進位格式，並不適合直接於前端進行即時編輯，因此系統於後端解析 MIDI 檔案，將其中之音符資訊轉換為結構化的 JSON 格式資料（notes JSON），再傳送至前端顯示。

此音符資料主要包含音高（pitch）、開始時間（start）與結束時間（end）等欄位，能清楚描述每一音符之時間位置與長度。

前端鋼琴捲軸（Piano Roll）編輯介面以 notes JSON 為基礎，將音符資料轉換為可視化的圖形元素，使使用者能以拖曳方式調整音符位置、音高與長度，或進行新增與刪除操作。此種設計可避免使用者直接操作 MIDI 檔案所帶來之

複雜性，並提升系統之操作直覺性與編輯彈性。

當使用者完成音符編輯後，前端會將更新後的 notes JSON 與相關檔名資訊，透過後端 API 傳送至伺服器進行後續處理。後端在接收資料後，會依據音符資料重新產生 MIDI 檔案，並執行單音化邏輯處理，以確保輸出之旋律符合單旋律表示方式，避免音符重疊對簡譜輸出造成影響。

在最終輸出階段，後端系統以 MIDI 檔案作為核心中介格式，進行後續之樂譜轉換流程。透過將 MIDI 轉換為 MusicXML，再進一步轉換為簡譜語法與 PDF 文件，可使系統同時保留機器可讀之 MIDI 資料，以及人類易於閱讀之簡譜文件，提升輸出結果之實用性與延展性。

整體而言，本系統透過 JSON 作為前後端主要資料交換格式，以 MIDI 作為音樂資料之中介格式，不僅降低前端實作複雜度，也使後端處理流程具備良好的模組化與可維護性，為後續功能擴充與系統優化提供彈性。

3.4 單音化方法設計

在音訊自動轉換與使用者於鋼琴捲軸介面進行編輯的過程中，可能產生多個音符於同一時間點同時發聲之情況，造成旋律線出現重疊。此類重疊音符若未經處理即進行輸出，將影響旋律表現的清晰度，亦可能導致後續簡譜排版結果難以閱讀。因此，本系統於後端設計並實作單音化處理模組，將音符資料轉換為任一時間點僅保留單一音符的單旋律表示方式，以符合簡譜輸出之需求。

單音化處理流程如圖 3.3 所示。系統首先讀取前端傳回之音符資料以 JSON

格式 (notes JSON) 表示，其中每一筆音符皆包含音高 (pitch)、開始時間

(start) 與結束時間 (end) 等資訊。為確保處理順序正確，系統會先依音符之開始時間進行排序，使後續比較能依時間先後進行。

排序完成後，系統以相鄰音符為比較單位，逐筆檢查目前音符與下一音符之時間區間是否存在重疊情形。當下一音符的開始時間早於目前音符的結束時間，即判定為音符重疊。針對此情況，系統採用截斷前一音符結束時間的方式進行修正，將前一音符之結束時間調整為下一音符之開始時間，使兩音符不再同時發聲；若相鄰音符間不存在重疊，則保留原有音符資料不作調整。

上述處理流程會反覆執行，直到所有音符皆完成檢查與修正。完成單音化處理後，系統即可產生不含重疊音符之單音音符序列，並依此重新輸出單音 MIDI 檔案，作為後續簡譜轉換與排版之輸入資料。

透過此單音化邏輯設計，本系統能有效避免音符重疊對簡譜輸出造成之影響，確保輸出旋律具備清楚的單旋律結構，提升最終簡譜 PDF 之可讀性與實用性。

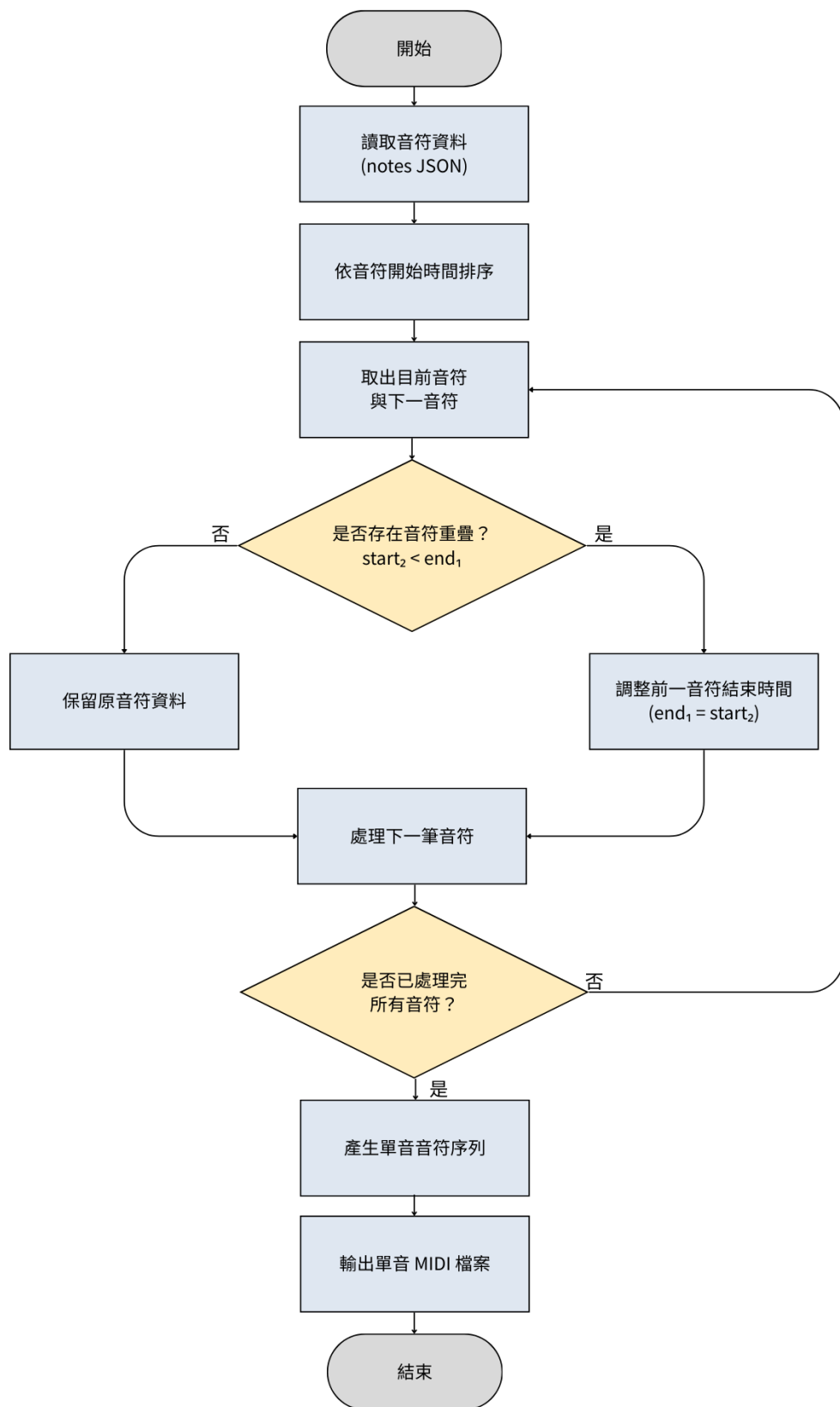


圖 3.3: 單音化流程圖

第四章

系統實作

4.1 整體架構

本系統採用前後端分離之處理流程。使用者於網站介面上傳 MP3 或 WAV 格式之音訊檔後，前端將檔案以 HTTP 請求送至後端 Flask 服務，並觸發後續轉換程序。後端首先使用 Demucs 進行音源分離，以取得人聲音軌並降低伴奏對音高偵測之干擾；接著透過 Basic Pitch 進行音高與起訖時間（onset/offset）之估測，將音訊轉譯為 MIDI 表徵。

考量自動轉譯結果可能產生音高誤判或節奏偏移，系統提供 MIDI 編輯介面供使用者進行人工校正。完成校正後，系統將 MIDI 事件轉換為本專題定義之簡譜語法 token 序列，輸出為文字格式，並以 LilyPond 進行排版渲染，生成可供下載之簡譜圖檔/PDF。

4.2 使用者介面實作

4.2.1 上傳介面

上傳介面以 HTML 搭配 CSS 實作，提供音訊檔案選擇與提交功能，並限制可上傳之檔案格式為 MP3 與 WAV。使用者提交後，前端將音訊檔傳送至後端 API，

以進行音源分離、音高轉譯與後續處理，並依回傳結果導引至編輯或輸出頁面。



面。

圖 4.1:上傳音檔介面

4.2.3 編輯音檔

音訊轉譯完成後，後端產生對應之 MIDI 檔案。由於 Basic Pitch 自動轉譯結果仍可能存在音高偏差、雜訊音符或節奏不精確等情形，前端將 MIDI 解析後以鋼琴卷軸（piano-roll）方式圖形化呈現，供使用者檢視與修正。介面中縱軸對應音高、橫軸對應時間，音符以矩形區塊表示其音高與持續時間。

為支援修正需求，系統提供下列操作模式：

1. 一般編輯模式：可拖曳音符區塊以調整音高或時間位置，並可拖曳音符左右邊界以修改音符持續時間；同時提供播放控制以即時檢核修正結果。
2. 刪除模式：可選取（點擊）音符區塊以移除不正確或多餘之音符事件。
3. 新增模式：可於指定時間與音高位置新增音符，以補足漏偵測之音符事

件。

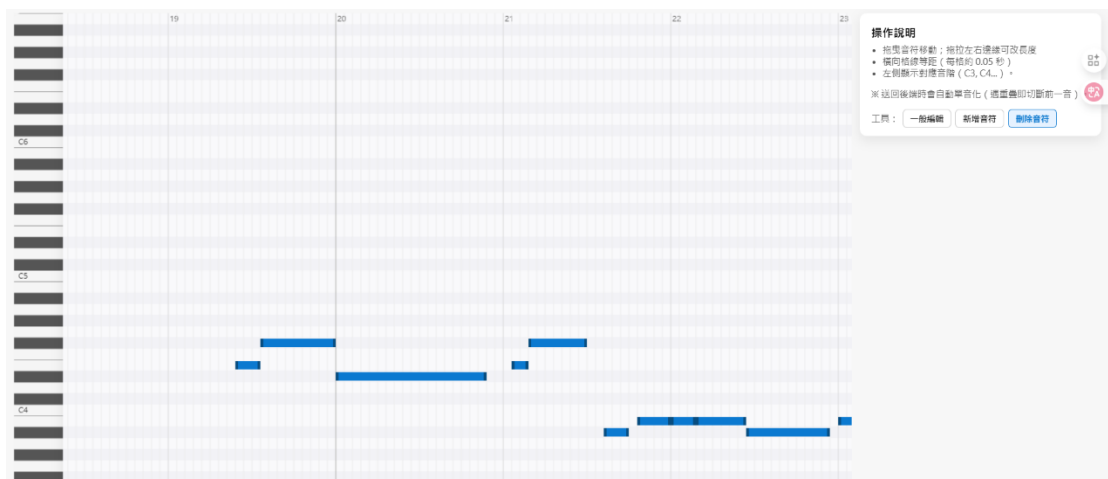


圖 4.2：MIDI 編輯介面（piano-roll）示意；左側鋼琴鍵盤用於音高對照與輔助校正。

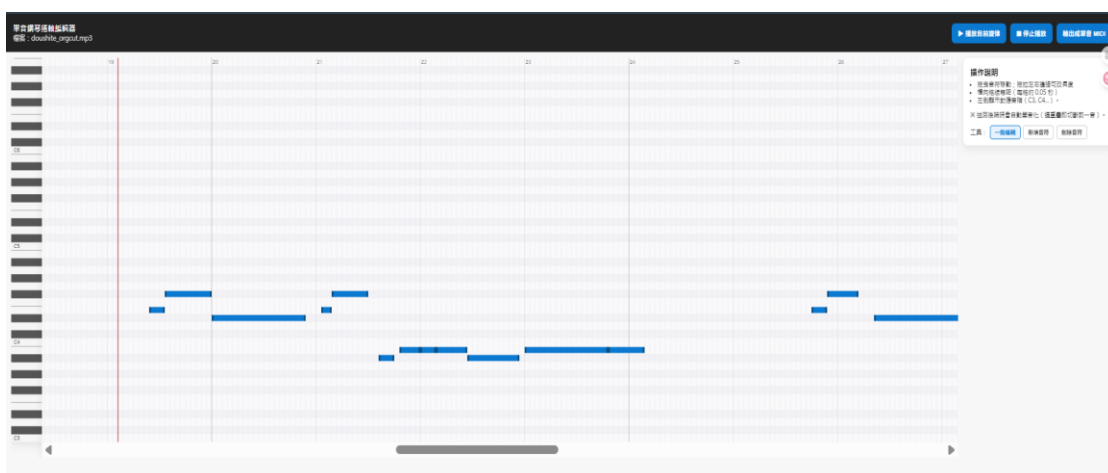


圖 4.3: 一般編輯模式之介面；紅色游標線表示目前播放/編輯之時間位置。

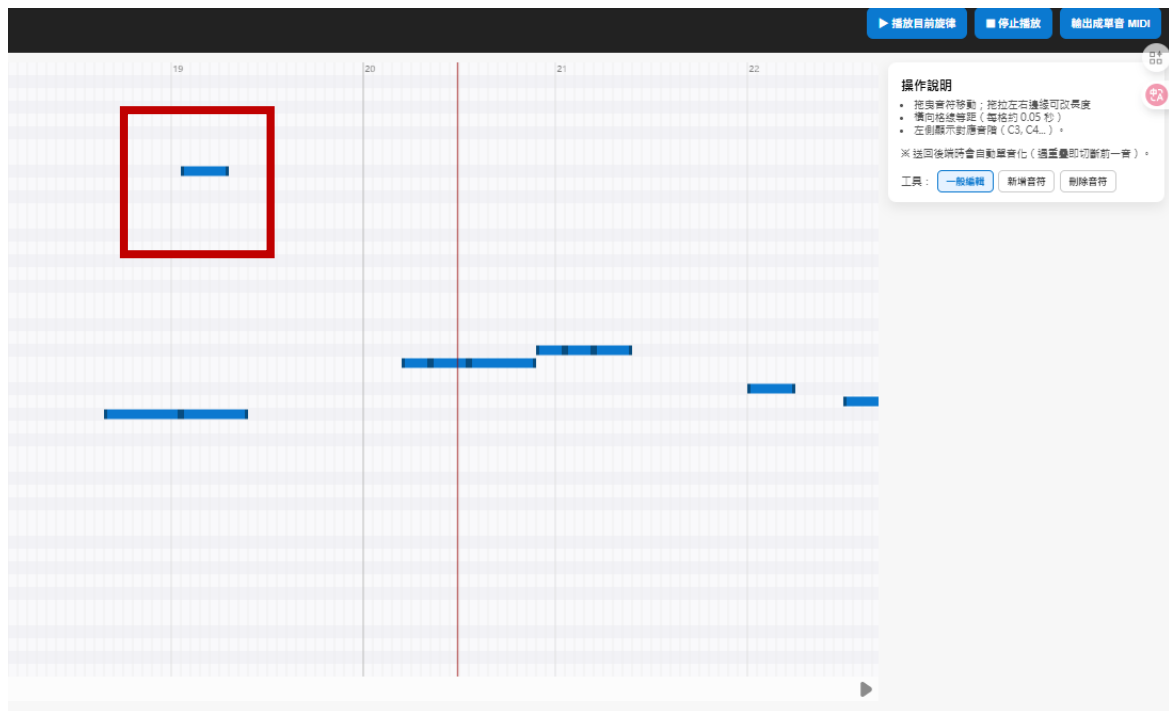


圖 4.4: 介面切換至刪除模式之示意。

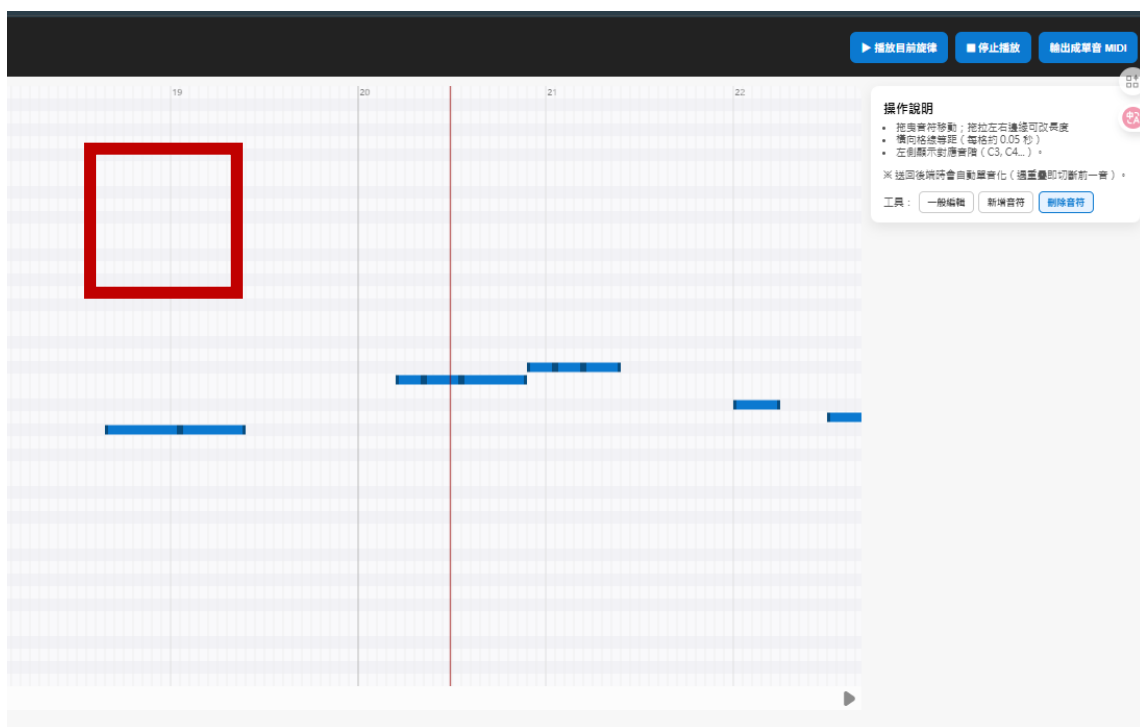


圖 4.5: 刪除模式操作示意：選取音符區塊以移除音符事件刪除模式。

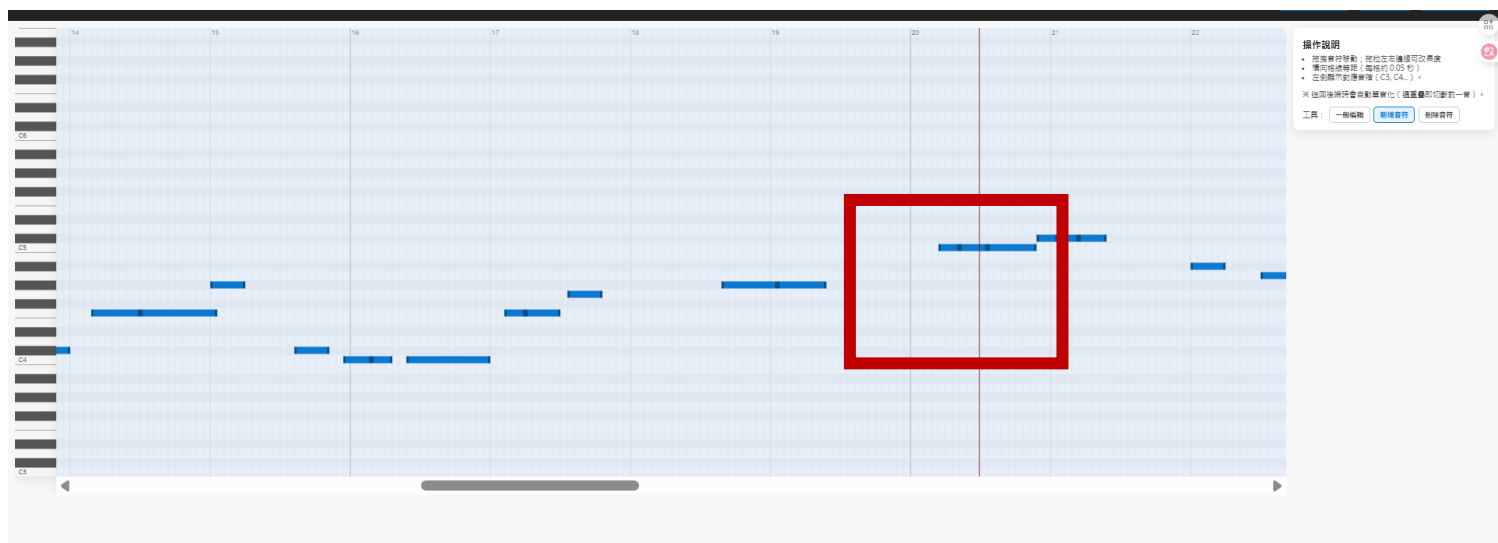


圖 4.6: 新增模式操作示意：於指定時間與音高位置新增音符事件。

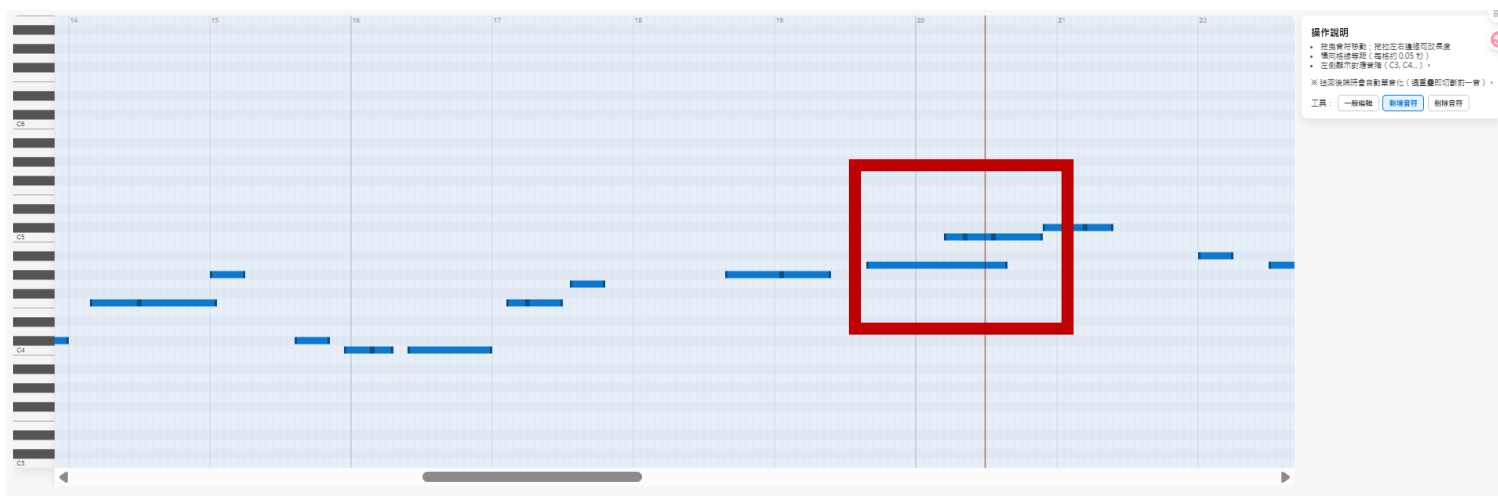


圖 4.7: 新增模式結果示意：新增音符後之音符分佈。

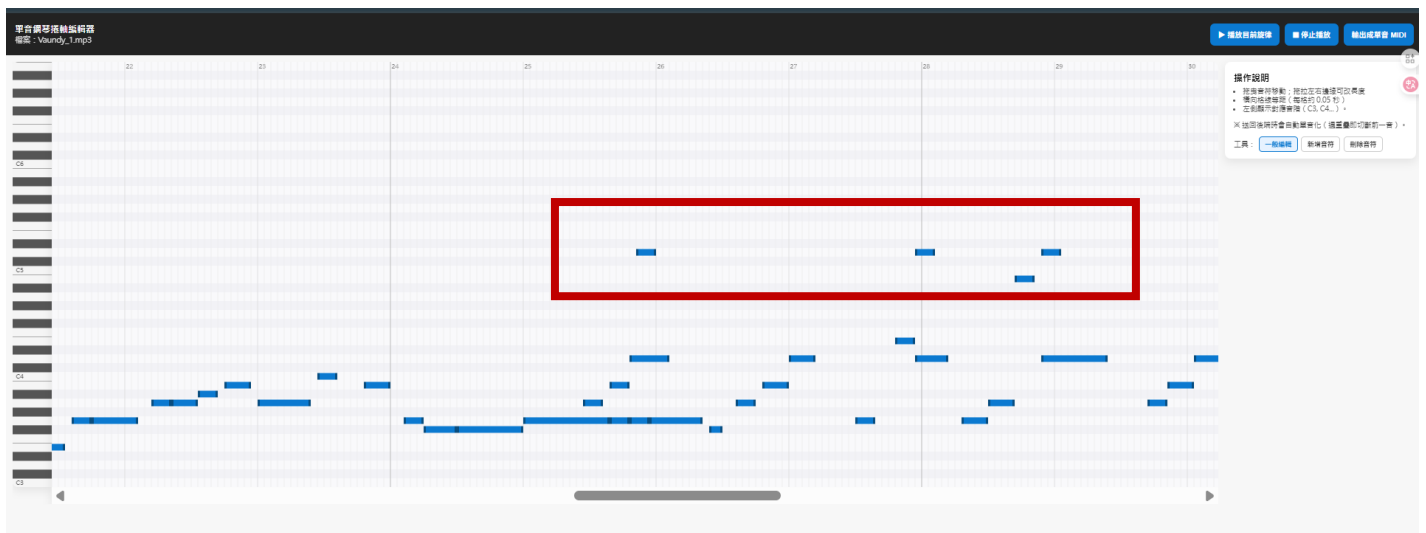


圖 4.8: 自動轉譯結果範例；紅框標示可能之誤判或多餘音符事件。

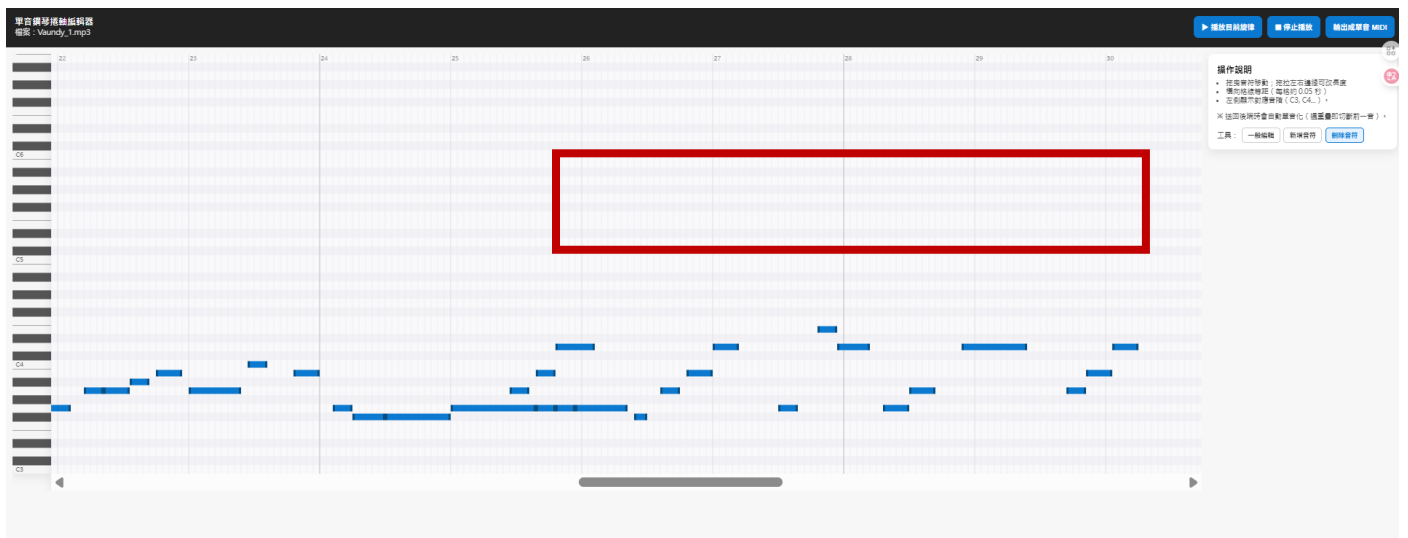


圖 4.9: 修正後結果示意；已移除圖4.8 所示之不正确音符事件。

4.2.4 結果呈現

系統在 MIDI 校正完成後，依 MIDI 音符事件生成簡譜語法並交由 LilyPond 進行排版渲染。前端提供輸出結果預覽區，以嵌入式檢視器呈現產出之簡譜（PDF/圖檔），並提供下載功能，使使用者可取得最終輸出檔案以利列印、保存或後續應用。

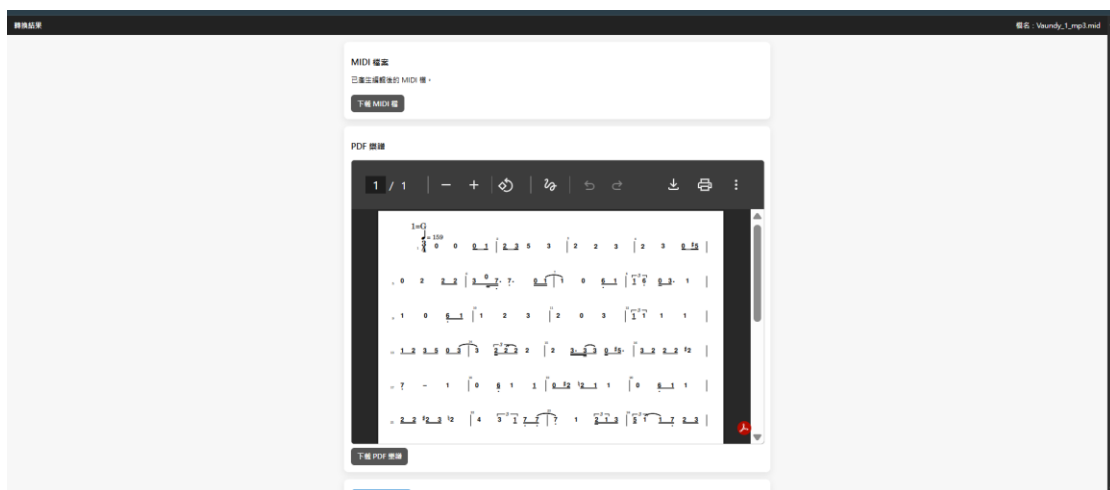


圖 4.10: 由修正後 MIDI 生成之簡譜輸出預覽（嵌入式檢視器）。



圖 4.11: 簡譜輸出檔案（PDF）之檢視示意。

4.3 後端處理流程實作

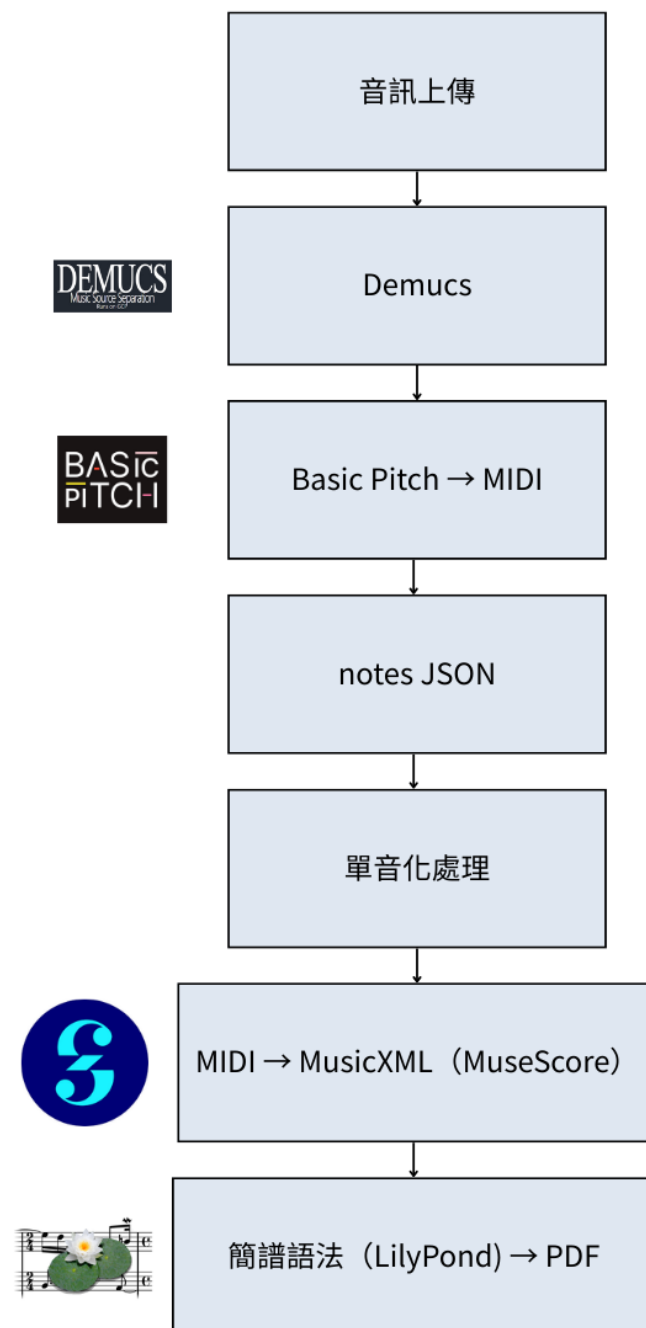


圖 4.12 :後端處理流程示意圖。

4.3.1 音訊接收與轉換處理

當使用者於前端上傳音訊檔案後，系統透過 HTTP 請求將音訊資料傳送至後端 Flask 伺服器。後端接收到請求後，首先進行檔案格式與完整性檢查，並將音訊檔案儲存至伺服器指定目錄，以利後續處理流程使用。

在完成檔案儲存後，系統依使用者設定判斷是否啟用人聲分離功能。若使用者選擇進行人聲分離，後端即呼叫 Demucs 模組對音訊進行處理，抽取其中之人聲音軌，以降低伴奏對後續音高偵測所造成的干擾；若未啟用此功能，系統則直接以原始音訊檔作為後續處理之輸入。

完成音源選擇後，系統將音訊資料送入 Basic Pitch 模組進行音高轉譯處理。Basic Pitch 會對音訊進行分析，推估音符之音高及其開始與結束時間，並輸出對應之 MIDI 檔案。此 MIDI 檔作為系統後續音符編輯與處理之基礎資料，並將於下一階段轉換為結構化之音符資料格式，以供前端顯示與使用者編輯。

4.3.2 音符資料處理與單音化實作

在完成音訊轉換並產生初始 MIDI 檔案後，系統進一步對 MIDI 內容進行解析，以取得可供前端編輯之音符資料。由於 MIDI 屬於二進位格式，不利於直接於網頁端進行操作與即時編輯，因此後端會先將 MIDI 中之音符事件轉換為結構化之 JSON 格式 (notes JSON)。該格式主要包含音符音高 (pitch)、開始時間 (start) 與結束時間 (end) 等資訊，使前端能以鋼琴捲軸 (Piano Roll) 形式直觀呈現音符內容。

使用者於前端完成音符編輯後，系統會將更新後之 notes JSON 與相關檔案識別資訊透過 API 傳回後端。後端接收到資料後，會依據 JSON 內容重新產生

對應之 MIDI 檔案，以反映使用者實際調整後的音符結果。此步驟確保前端編輯內容能正確同步至後端，並作為後續處理流程之輸入。

為避免音符重疊對簡譜輸出造成影響，系統於重新產生 MIDI 後，會執行單音化處理流程。單音化模組會依音符開始時間進行排序，並逐一檢查相鄰音符之時間區間是否存在重疊情形；若判定為重疊，則透過調整前一音符結束時間的方式進行修正，使任一時間點僅保留單一音符發聲。完成單音化處理後，系統即可輸出符合單旋律條件之 MIDI 檔案，作為後續簡譜轉換與排版之基礎資料。

4.3.3 簡譜轉換與輸出

在完成單音化處理並取得符合單旋律條件之 MIDI 檔案後，系統即進入簡譜轉換與輸出階段。由於 MIDI 格式主要用於音樂資料交換，並不適合直接進行樂譜排版，因此本系統以 MIDI 作為中介格式，逐步轉換為適合簡譜輸出的表示形式。

首先，後端透過 MuseScore 提供之指令列介面，將單音 MIDI 檔案轉換為 MusicXML 格式。MusicXML 為結構化的音樂描述格式，能完整保留音符、節奏與小節等資訊，適合作為後續譜面轉換與處理之中介資料。完成轉換後，系統再依據 MusicXML 內容，將音樂資訊轉譯為本專題所採用之簡譜語法表示。

在取得簡譜語法後，系統進一步呼叫 LilyPond 排版工具進行譜面渲染。

LilyPond 具備高品質音樂排版能力，能依簡譜語法自動產生版面整齊且可讀性良好之樂譜輸出。最終，系統將渲染完成之簡譜以 PDF 格式輸出，作為使用者下載與保存之成果檔案。

完成上述流程後，後端會將產生之簡譜 PDF 檔案與對應之單音 MIDI 檔案整理並回傳至前端結果頁面，供使用者進行線上預覽與下載。透過此一轉換與輸出機制，本系統能將音訊資料完整轉換為具備實用價值之簡譜成果，完成整體音訊轉簡譜之處理流程。

第五章

結論與未來發展

5.1 結論

本專題成功地設計並實現了一個自動化的「音訊到數字簡譜轉譯系統」，其核心價值在於將複雜的音樂處理流程整合為一個高效、一鍵式的 Web 應用。本研究的成果集中體現於跨領域技術的無縫串接，以及對成熟開源工具的創新整合應用，成功建立了一套端到端（End-to-End）的智慧化製譜解決方案。

系統整合了深度學習音訊處理、標準化樂譜交換格式與專業排版引擎三大核心環節：首先，利用 Hybrid Demucs [1] 確保了高品質的人聲分離，有效去除了背景音樂對轉譯的干擾，為後續處理提供了純淨的音源；接著，透過 Basic Pitch [2] 進行輕量且高效的音高與時值轉譯，產出初步的 MIDI 資料。然而，由於 AI 模型生成的原始 MIDI 訊號往往伴隨著微小的時間誤差（Timing Jitter），若直接進行樂譜編碼，容易導致節奏判讀混亂或產生過多不必要的休止符。

針對此一關鍵的樂譜生成階段，本研究採取了與傳統手動製譜截然不同的自動化策略。儘管業界廣泛使用的 MuseScore [3] 多被視為圖形使用者介面（GUI）工具，但本專題深入挖掘並利用了其強大的指令列介面（Command Line Interface, CLI）功能，將其轉化為後端自動化管線中的核心轉檔引擎。系統利用 MuseScore 內部成熟且經過驗證的量化演算法（Quantization Algorithm），將 Basic Pitch 產出的原始 MIDI 檔案自動進行時值校正，並轉換為具備完整樂理結構的 MusicXML 標準格式。此步驟至關重要，它將非結構化的音符觸發訊號，轉化為包含正確小節劃分、拍號定義與音符時值的結構化資料。

隨後，基於標準化的 MusicXML 資料，本研究整合了樂譜轉碼模組，將 XML 資訊精確解析並映射為 jianpu-ly [4] 可識別的簡譜語法，最終交由 LilyPond [5] 引擎進行專業渲染。這種「AI 轉譯 + MuseScore 標準化 + LilyPond 排版」的複合式架構，成功克服了「AI 原始數據」與「印刷級排版」之間的鴻溝，既利用了 MuseScore 強大的樂譜解析能力，又充分發揮了 LilyPond 輸出高品質 PDF 的優勢，實現了從數位訊號到可視化樂譜的完美轉換。

本系統介面設計簡潔直觀，使用者只需上傳音檔並輸入 BPM 與拍號即可獲得目標簡譜，這直接解決了音樂教育、教材製作等應用場景中，手動製譜所產生的高昂時間與金錢成本，完全達成了本專題「利用 AI 達到省錢省時間的成品效益」的動機。總結而言，本專題為有教材需求的音樂工作者提供了一套經過驗證、高效且具備高度擴展性的解決方案，不僅展示了資訊工程技術在音樂內容創作領域的巨大潛力，更為未來的自動化採譜研究提供了具參考價值的實作架構。

5.2 未來發展方向

為了進一步提升本系統的性能、用戶體驗與應用廣度，未來的研究與發展可從以下三個主要方向深入探索：

首先，應著重強化 MIDI 轉譯的準確性與細節呈現。雖然在 Basic Pitch 泛用性上表現優異，但在轉譯的音符邊緣與節奏細節上仍有提升空間。未來可考慮開發基於音樂理論規則的後處理（Post-Processing）機制，以修補 MIDI 輸出中常見的音符碎片，提升簡譜的節奏準確度。同時，應研究如何從 Basic Pitch 提供的 pitch bend 或 activation 資訊中，自動提取並標註簡譜中的力度符號或表情記號，使產出的樂譜更貼近實際演奏的細節。

其次，擴展簡譜編碼的複雜度與彈性是系統進階的關鍵。目前的簡譜編碼主要針對單一旋律線進行。未來可引入和弦識別（Chord Recognition）模組，將 Demucs 分離出的 Bass 或 Others 軌道轉換為簡譜上方的和弦記號，以大幅提升教材的實用性。此外，系統應發展支援多聲部或多樂器簡譜的能力，允許將分離出的多個音源軌道轉換成多行簡譜，以支持複雜合奏樂譜的自動生成。

最後，優化系統的部署與使用者互動性將是提升實務可用性的重要方向。這包括探索使用 ONNX 等格式對 Demucs 與 Basic Pitch 進行模型壓縮與優化，並利用 GPU/TPU 資源加速推論，以降低系統的推論延遲。更重要的是，在前端結果呈現層加入基本的所見即所得（WYSIWYG）編輯器，允許使用者在下載 PDF 前，對自動生成的簡譜進行手動修正（例如調整錯別音高、修改拍號），這將能有效彌補 AI 轉譯的最終誤差，提供更完整、更人性化的使用體驗。

參考文獻

- [1] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "Hybrid spectrogram and waveform source separation," arXiv:2111.03600v2, Nov. 2021. [Online]. Available: <https://arxiv.org/abs/2111.03600>
- [2] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert, "A lightweight instrument-agnostic model for polyphonic note transcription and multipitch estimation," *arXiv preprint arXiv:2203.09893v2*, 2022.
- [3] MuseScore, "MuseScore: Free music composition and notation software," Ver. 4.0. [Online]. Available: <https://musescore.org>. [Accessed: Dec. 02, 2025].
- [4] ssb22, "jianpu-ly.py: Render Jianpu notation with LilyPond," GitHub, 2023. [Online]. Available: <https://github.com/ssb22/jianpu-ly>
- [5] H. W. Nienhuys and J. Nieuwenhuizen, "LilyPond, a system for automated music engraving," in *Proc. Colloquium on Musical Informatics (XIV CIM 2003)*, May 2003.